LOHA: A Latency-Optimized CPU-Storage Hybrid Architecture for Billion-Scale Graph-based Vector Similarity Search

Seongjoon Cho[1], Junyoung Park[1], Donghyun Kang[2], Moohyeon Nam[2], Hongchan Roh[2], Se-Hyun Yang[2], Moo-Kyoung Chung[2] and Seungkyu Choi[3]
[1]Kyung Hee University [2]Dnotitia Inc. [3]Yonsei University, South Korea

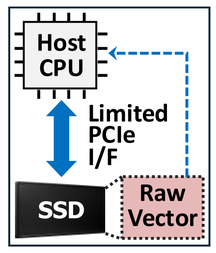
1. Introduction & Motivation

▪ **Target Algorithm: Vector Similarity Search**

➢ Vector similarity search retrieves data semantically similar to a given query.

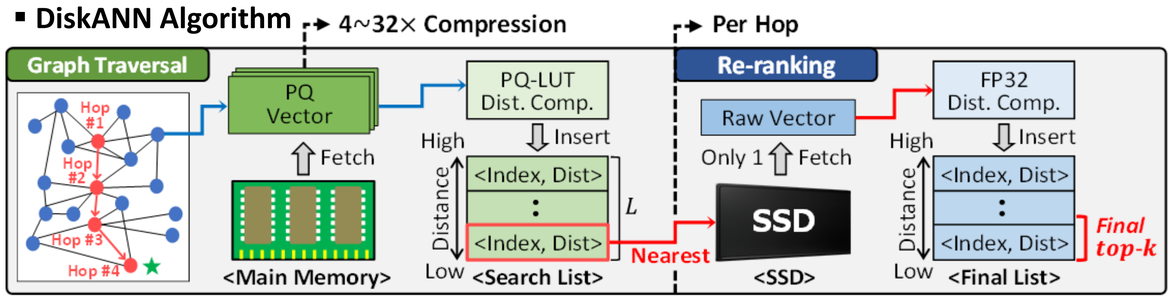
▪ **Graph Capacity Requirements across ANNS Benchmark Datasets**

	Deep 1B	T2I 1B	LAION 100M	YFCC 100M	OpenAI-I 10M
Data Type	FP32	FP32	FP32	FP32	FP32
Dimension	96	200	768	1280	3072
Neighbor Index	132GB	132GB	13.2GB	13.2GB	1.3GB
Raw Vector	400GB	800GB	307GB	512GB	123GB
Overall	532GB	932GB	320.2GB	525.2GB	124.3GB



➢ In billion-scale search, raw vectors reside on SSDs and are accessed via PCIe I/F.

▪ **DiskANN Algorithm**

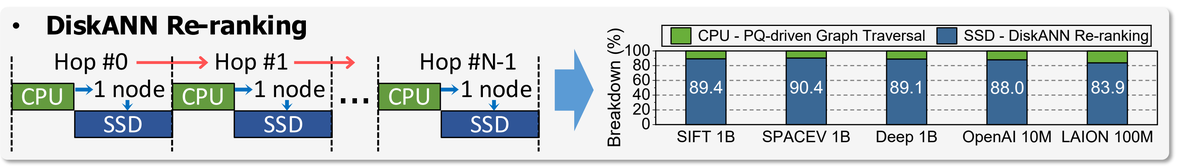


≅ **92~95% of Total Comp.** ≅ **5~8% of Total Comp.**

➢ DiskANN reduces SSD accesses to 5–8% via compression while preserving recall.

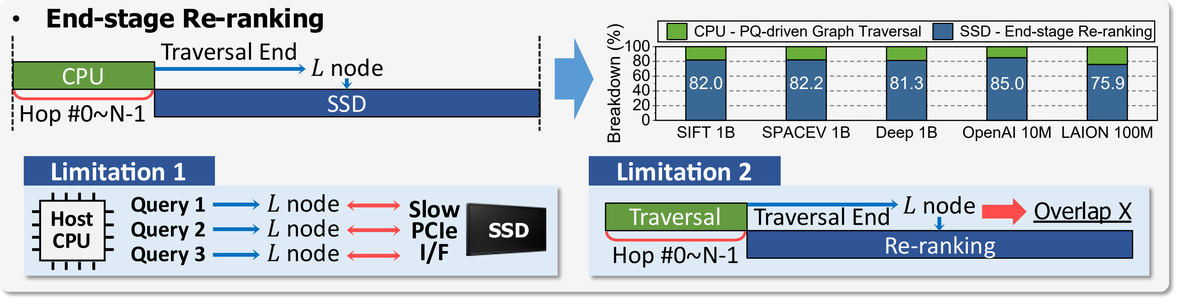
▪ **Motivation**

• **DiskANN Re-ranking**



SSD-level Parallelism↑ Re-ranking Latency Slightly↓

• **End-stage Re-ranking**

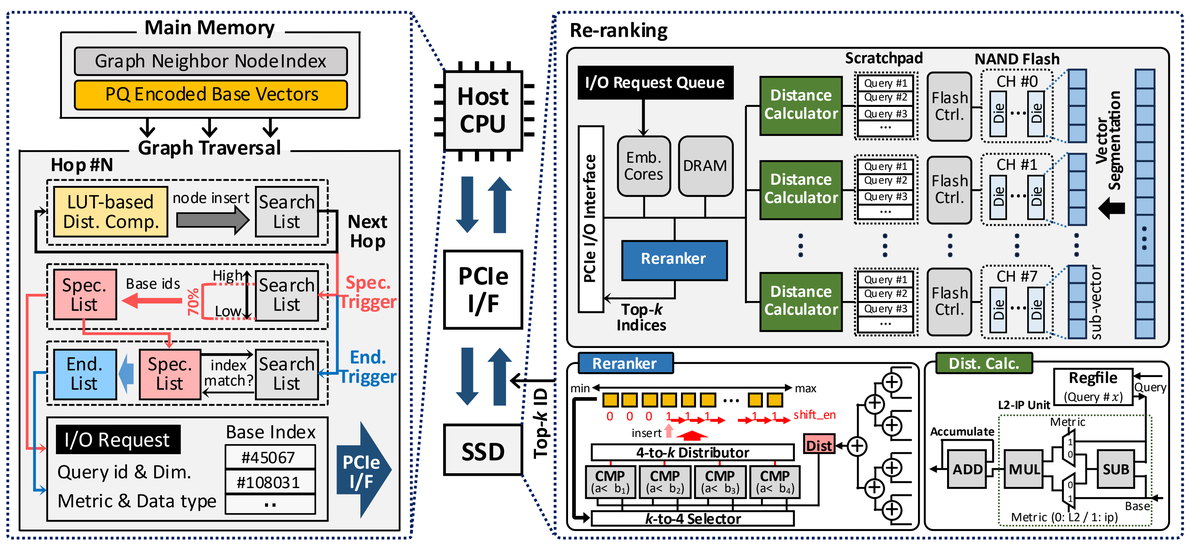


➢ **Limitation 1**: Batched SSD accesses lead to PCIe bandwidth saturation.

➢ **Limitation 2**: Re-ranking starts only after graph traversal, preventing workload overlap.

2. Proposed Architecture

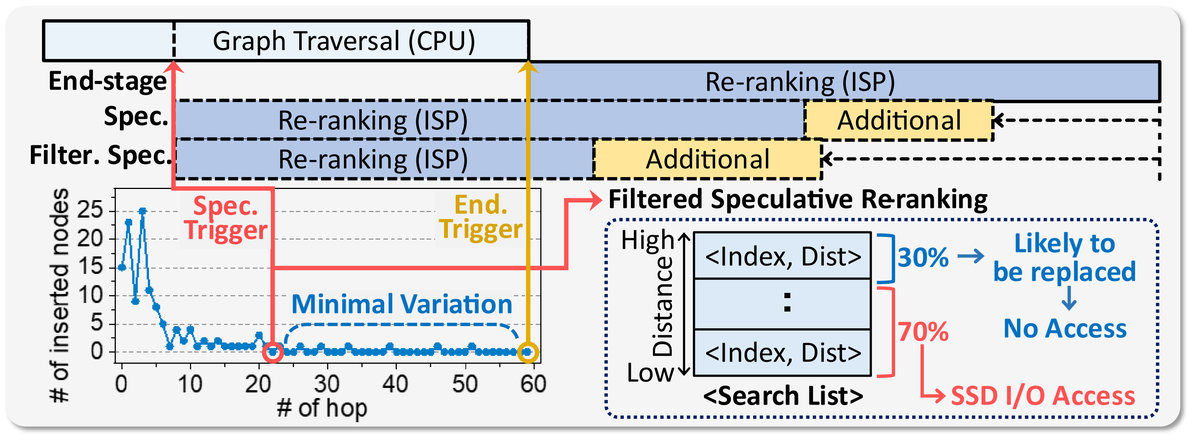
▪ **LOHA: CPU-SSD Hybrid Architecture**



➢ **CPU-Side**: CPU performs fast LUT distance computation using compressed PQ vectors. (PQ Compression Ratio: 4–32×, PQ-LUT Speedup over Baseline: 1.7–8.9×)

➢ **SSD-Side**: Each channel includes a PE, and the reranker aggregates the results. (Mitigated PCIe Bottlenecks & Reduced NAND Flash Reads & Lightweight Modules)

▪ **Speculative Re-ranking**



	Speculative Trigger	Speculative Re-ranking		Filtered Speculative Re-ranking	
		Matching (%)	SSD I/O	Matching (%)	SSD I/O
L50	Hop 22.0 (**36.6%**)	**66.48%**	67	63.02% (−**3.5%**)	53 (−**20.9%**)
L100	Hop 31.2 (**28.4%**)	**66.00%**	134	62.49% (−**3.5%**)	107 (−**20.1%**)
L200	Hop 45.1 (**21.3%**)	**65.41%**	269	62.33% (−**3.1%**)	215 (−**20.1%**)

➢ Prediction exploits the minimal changes in the search list during the later hops.

➢ Early prediction (20–30%) with ~65% matching accuracy enables latency reduction.

➢ Applying additional filtering at the speculative trigger reduces SSD I/O by ~20%.

3. Evaluation Results

▪ **Evaluation Methodology**

Dataset	Data type	*Dim.*	Size	Metric	PQ Comp.
SIFT 1B	uINT8	128	125GB	L2	4×
SPACEV 1B	INT8	100	137GB	L2	4×
Deep 1B	FP32	96	375GB	L2	12×
OpenAI 10M	FP32	3072	120GB	L2 / IP	32×
LAION 100M	FP32	768	306GB	L2 / IP	32×

System	Configuration
CPU	2 sockets, 2.8GHz 32 cores / 64 threads
SSD	3.84TB, 8 channels, 1.2GB/s channel B.W, Flash RD latency 22.5us

Simulation of Re-ranking in SSD

① Implement the end-stage re-ranking in C++ using *libaio* (API for async I/O).

② Capture I/O traces using *blktrace*.

③ Fed into *MQSim* to simulate SSD-level behavior. (+cycle-accurate sim.)

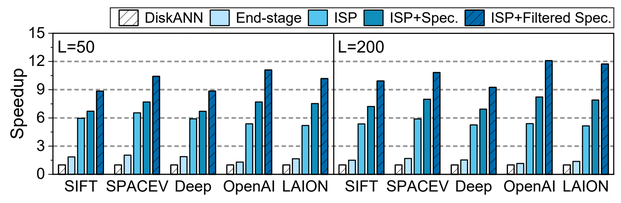
Comparison Baselines

DiskANN, SPANN	SSD-based open-source implementations
GGNN	GPU-based ANNS implementation (4 A6000 GPUs)
NDSEARCH, REIS	End-to-End in-storage accelerator (SOTA)
FusionANNS	CPU/GPU-SSD collaborative architecture (SOTA)

▪ **In-Storage Accelerator Overheads (TSMC 28nm, 300MHz)**

Modules		Area (mm^2)	Power (mW)
Channel-level Distance Calculator	Reg. File	0.0381	14.4
	L2-IP Unit	0.0023	0.6
	Accumulator	0.0018	1.0
SRAM Scratchpad 96KB		0.1112	41.2
Total		0.1537	57.3
SSD-level Re-ranker	Adder Tree	0.0064	0.7
	Sorter	0.0265	8.3
Total		0.0334	9.23

▪ **Re-ranking Latency Analysis**



➢ Re-ranking latency gradually improves.

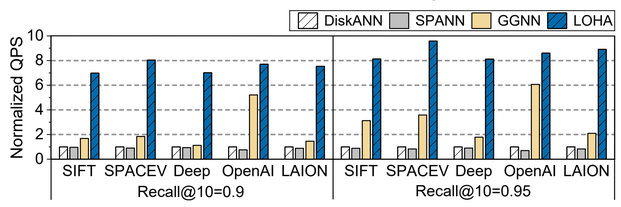
✓ SSD internal parallelism (up to 2.0×)

✓ PCIe transfer minimization (up to 4.7×)

✓ Workload overlap (up to 1.5×)

✓ SSD I/O filtering (up to 1.5×)

▪ **End-to-End Performance Analysis**

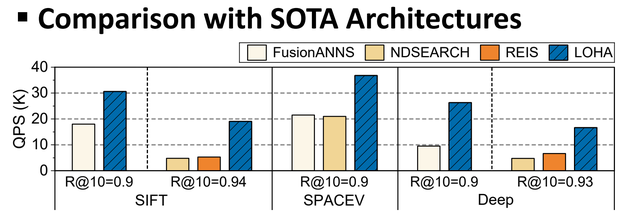


➢ LOHA achieves higher performance.

✓ DiskANN, SPANN (up to 9.6×, 12.0×) (Optimizes SSD workload.)

✓ GGNN (up to 6.2×) (Eliminates CPU-GPU transfer overhead.)

▪ **Comparison with SOTA Architectures**



➢ LOHA achieves higher performance.

✓ FusionANNS (up to 2.8×) (Minimizes data movement.)

✓ NDSEARCH, REIS (up to 4.0×, 3.6×) (Substantially reduce page reads.)

4. Conclusion

▪ We develop a hybrid design that combines software-level compression and in-storage processing to optimize both CPU and SSD workloads.

▪ A speculative re-ranking mechanism leverages the intrinsic properties of GANNS to overlap CPU and SSD operations through prediction.

▪ Up to 9.6×, 6.2×, 4.0× higher speedup over the CPU-, GPU- baselines and in-storage architectures, respectively, with minimal in-storage hardware overhead.

Ultimately, LOHA demonstrates the effectiveness of our hybrid architecture for scalable, low-latency billion-scale vector search, enabling efficient acceleration without relying on bespoke accelerators.